BDD SQL — Devoir 4

BUT3 GEA — IUT de Saint-Denis

Avant de commencer

Jusqu'ici, vous avez seulement **lu** des bases de données. Dans ce devoir, vous allez **modifier votre base individuelle** (celle des Devoirs 2 et 3) : créer une nouvelle table, ajouter, modifier et supprimer des lignes. Ensuite, vous l'interrogerez avec des jointures sur trois tables.

> **Exercices notés : 1 à 16.** Les exercices 17 et 18 sont des bonus.

> **Tout le devoir (parties A et B, bonus compris) se fait sur VOTRE base** : le fichier qui porte **votre numéro étudiant**, par exemple `12345678.db`, la même que pour les Devoirs 2 et 3. Pas sur chinook, pas sur la base d'un ou d'une camarade. Chaque base a des données différentes, et vos réponses sont vérifiées sur la vôtre : un devoir fait sur une autre base compte comme faux.

> **Où trouver votre base ?** https://github.com/frey-iutsd/sql-2026, dossier `sql2`, puis `bdd` (le lien ouvre directement ce dossier) : cliquez sur le fichier qui porte votre numéro étudiant, puis sur « Download raw file ». C'est aussi là que vous la retéléchargerez si vous devez repartir de zéro.

Les **notes de cours** (`notes-S1-S4.pdf`) donnent la syntaxe de toutes les commandes utilisées ici : gardez-les ouvertes.

> **Pas de SQLiteStudio ?** Tout le devoir peut se faire sur **https://sqliteonline.com/**, dans le navigateur, sans rien installer (aussi sur Mac). Menu **File → Open DB**, puis choisissez votre fichier `.db`. **Attention :** le bouton Run exécute tout ce qui est dans la zone de texte. Pour n'exécuter qu'une commande, sélectionnez-la avant de cliquer sur Run.

Comment rendre votre travail : comme pour les Devoirs 2 et 3.

1. Pour chaque exercice, écrivez une ligne `-- exercice N`, suivie de votre réponse.
2. Si la réponse est une commande SQL (`CREATE`, `INSERT`, `UPDATE`, `DELETE`, `SELECT`…), **exécutez-la d'abord** sur votre base, puis copiez-collez-la telle quelle, terminée par un `;`.
3. Si la réponse n'est pas une commande SQL (un nombre, un nom), écrivez-la précédée de `--`.
4. Enregistrez le fichier sous le nom **Prénom-NOM-SQL-Devoir4.txt** et déposez-le sur Teams, puis cliquez sur **Rendre**.

> **Votre fichier sera exécuté en entier, dans l'ordre, sur une copie neuve de votre base.** Il doit donc contenir **toutes** les commandes de la partie A, chacune **une seule fois**, dans l'ordre des exercices. Si vous vous trompez en cours de route, le plus simple est de repartir de votre base d'origine (retéléchargez-la sur GitHub, voir plus haut) et de réexécuter votre fichier corrigé. **Avant de rapporter vos nombres, repartez de votre base d'origine et exécutez votre fichier une seule fois, en entier.**

Partie A — Modifier votre base

On veut classer les produits en trois **catégories** : Papeterie, Informatique et Accessoires. Pour cela, on crée une table `categories`, puis on relie chaque produit à sa catégorie par une **clé étrangère**.

Exercice 1. Créez une table `categories` avec deux colonnes : `id`, un nombre entier qui est la **clé primaire**, et `nom`, un texte obligatoire (`NOT NULL`).

Exercice 2. En **une seule** commande `INSERT`, ajoutez les trois catégories suivantes dans la table `categories` : `1` Papeterie, `2` Informatique, `3` Accessoires.

Exercice 3. Ajoutez à la table `produits` une colonne `categorie_id`, de type nombre entier, qui est une **clé étrangère** vers `categories(id)`. (Commande `ALTER TABLE … ADD COLUMN`, voir les notes.)

Exercice 4. Avec **trois** commandes `UPDATE` (une par catégorie, en utilisant `IN`), remplissez la colonne `categorie_id` de chaque produit d'après cette liste. Votre base ne contient que 15 de ces 24 produits : ce n'est pas grave, écrivez quand même la liste complète.

- **Papeterie (1)** : Cahier, Stylo, Classeur, Agenda, Calculatrice, Trousse, Surligneur, Post-it, Agrafeuse
- **Informatique (2)** : Clé USB, Casque audio, Chargeur, Souris sans fil, Clavier, Enceinte Bluetooth, Ecouteurs, Powerbank, Webcam
- **Accessoires (3)** : Sac à dos, Gourde, Lampe de bureau, Tapis de souris, Support téléphone, Housse ordinateur

Écrivez les noms exactement comme ci-dessus, accents compris.

Exercice 5. Donnez une requête affichant tous les produits de la catégorie Informatique. (Vérifiez aussi, en affichant toute la table `produits`, que plus aucun produit n'a une catégorie vide.)

Exercice 6. D'après le résultat de l'exercice 5 : combien de produits de votre base sont dans la catégorie Informatique ?

Exercice 7. Ajoutez-vous comme cliente ou client dans la table `clients` : votre nom de famille comme `nom`, et `'IUT'` comme `ville`. Ne donnez pas de valeur pour `id` : SQLite choisit tout seul le numéro suivant, `92`.

Exercice 8. Ajoutez dans la table `commandes` une commande passée par ce nouveau client (`client_id` 92), pour le produit d'`id` 1, et dont le montant est égal à **2 fois le prix** de ce produit (regardez son prix dans la table `produits`, et écrivez directement le résultat du calcul). Ne donnez pas de valeur pour `id`.

Exercice 9. Le fournisseur augmente ses prix : augmentez de 10 % le prix de tous les produits de la catégorie Accessoires (et seulement ceux-là), arrondi à 2 chiffres après la virgule.

Exercice 10. Supprimez toutes les commandes dont le montant est **strictement inférieur à 25**.

Exercice 11. Donnez une requête qui compte les commandes restantes après l'exercice 10, puis, en commentaire, le nombre obtenu.

Attention avec UPDATE et DELETE. Sans `WHERE`, la commande s'applique à **toutes** les lignes de la table. Avant d'exécuter un `UPDATE` ou un `DELETE`, testez d'abord votre condition avec un `SELECT … WHERE …` pour voir quelles lignes seront touchées.

Partie B — Interroger la base modifiée

Exercice 12. Donnez une requête affichant, pour chaque commande, son `id`, le nom du produit, le nom de sa catégorie et le montant de la commande. (Jointure sur **trois** tables : `commandes`, `produits` et `categories`.)

Exercice 13. Donnez une requête affichant, pour chaque catégorie, son nom, le nombre de commandes (colonne `nb_commandes`) et le chiffre d'affaires arrondi à 2 chiffres après la virgule (colonne `ca`), triés par chiffre d'affaires décroissant. Regroupez par `categories.id` et `categories.nom`.

> **Exercice 14.** D’après le résultat de l’exercice 13 : quelle catégorie rapporte le plus, et combien ?

> **Exercice 15.** Donnez une requête affichant les villes dont les clients ont passé **au moins 3 commandes** de produits de la catégorie Informatique, avec ce nombre de commandes. (Jointure entre `clients`, `commandes` et `produits` ; il faut à la fois un `WHERE` et un `HAVING`.)

> **Exercice 16.** D’après le résultat de l’exercice 15 : combien de villes sont dans ce cas ?

Bonus

> **Exercice 17 (bonus).** Donnez une requête affichant, pour chaque couple (ville, catégorie), le chiffre d’affaires réalisé dans cette ville pour cette catégorie (arrondi à 2 chiffres), en ne gardant que les couples dont le chiffre d’affaires dépasse 1000, triés par chiffre d’affaires décroissant. (Quatre tables ; on regroupe par deux colonnes.)

> **Exercice 18 (bonus).** D’après le résultat de l’exercice 17 : combien de couples (ville, catégorie) sont dans ce cas ?

N’oubliez pas de déposer votre fichier `Prénom-NOM-SQL-Devoir4.txt` sur Teams et de cliquer sur « Rendre ».